4. Large Language Model Pretraining

CSC3043S Reasoning in AI: Large Language Models

Dr Jan Buys

University of Cape Town

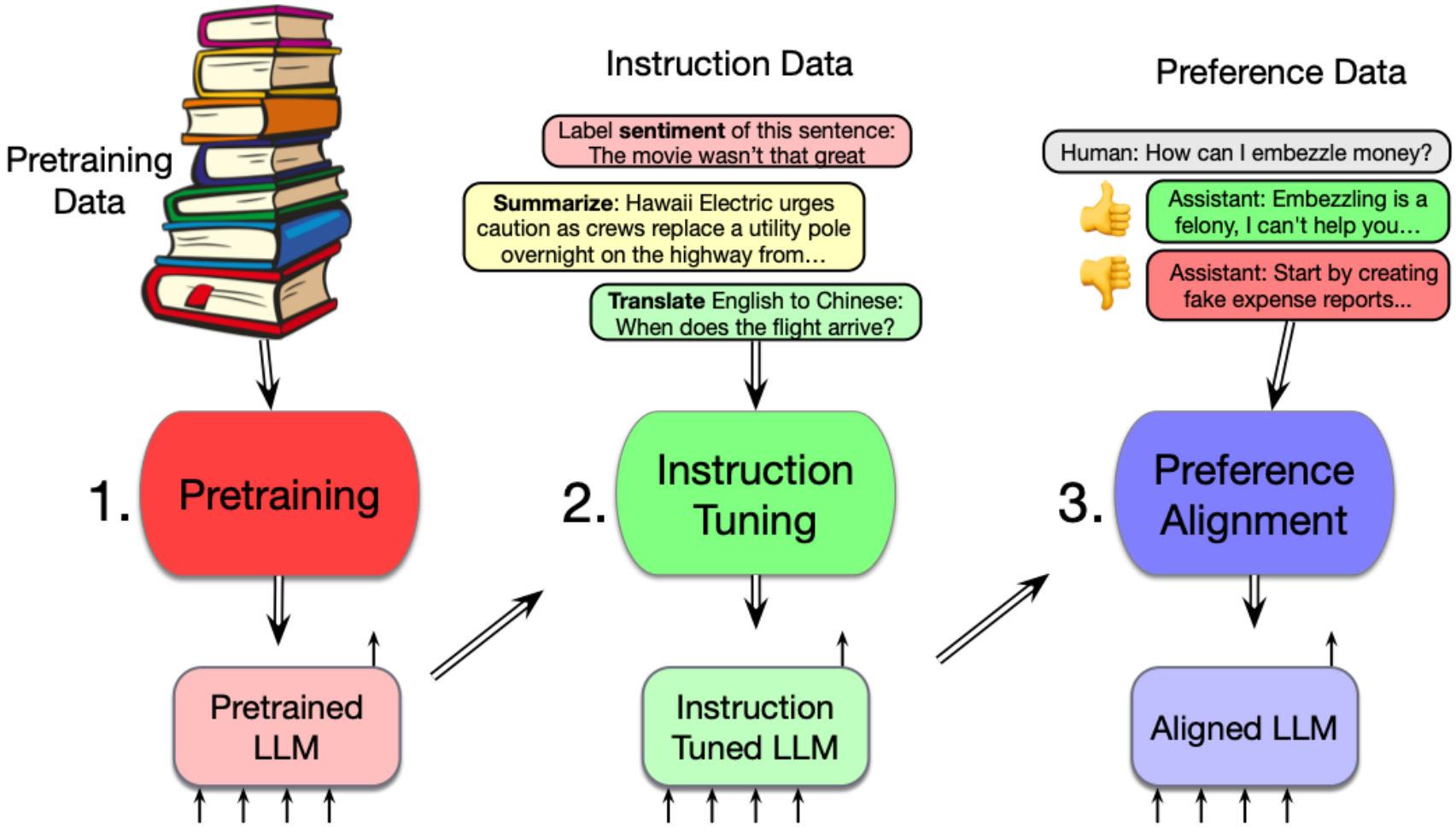
Slides from Jurafsky and Martin, Tatsunori Hashimoto (Stanford), others used as cited.



UNIVERSITY OF CAPE TOWN

IYUNIVESITHI YASEKAPA • UNIVERSITEIT VAN KAAPSTAD

Three stages of LLM training

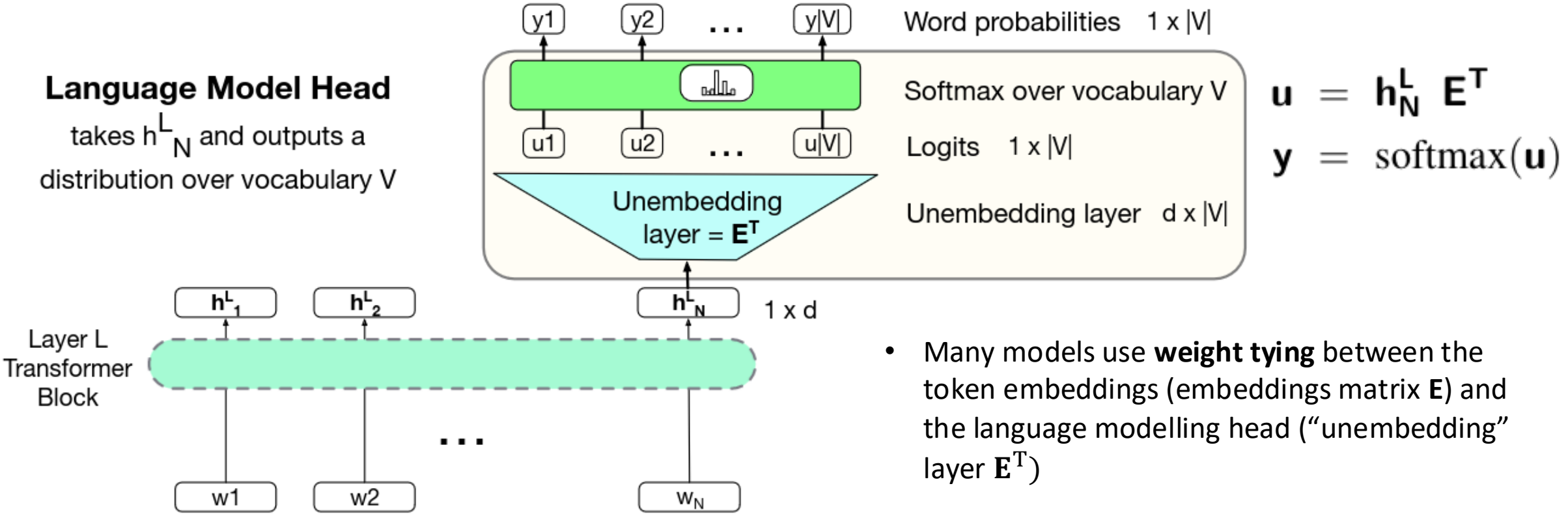


LLM Pretraining

- A Transformer language model is first **pretrained** on large amounts of text, then fine-tuned for instruction following
- Learning to predict the next word is a form of *self-supervised* learning because it uses the next word as label

LLM Pretraining

- Language modeling head: predicts the next word distribution



- Many models use **weight tying** between the token embeddings (embeddings matrix **E**) and the language modelling head ("unembedding" layer $\mathbf{E}^{\mathrm{T}}$)

Jurafsky and Martin (2024) Ch. 10

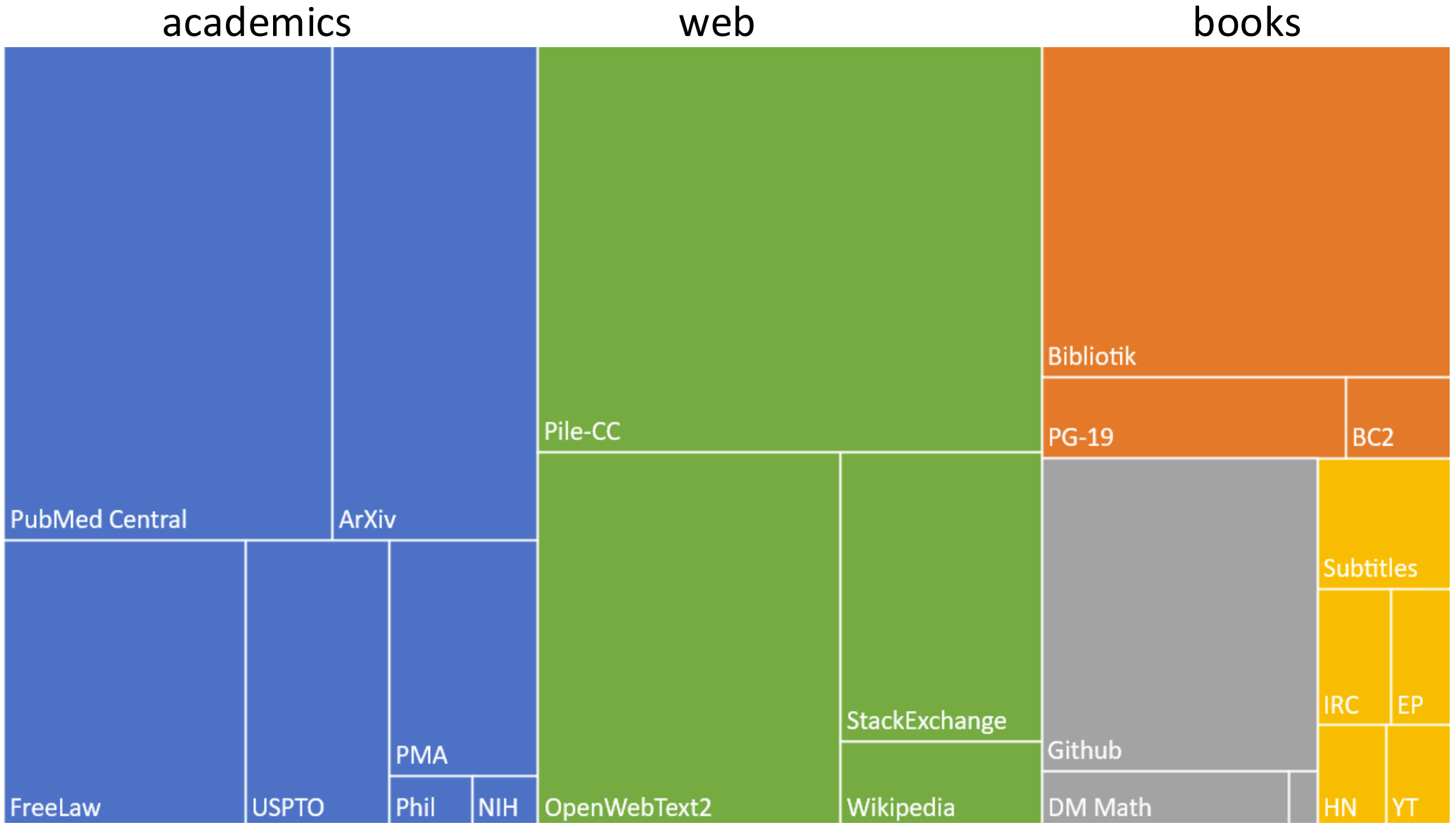
LLM Pretraining

- Pretraining data is the basis for developing LLMs
- “Scaling laws” show that model performance is a function of the pretraining data size, model size, and training time
- Large open-source LLMs are now typically trained on trillions tokens of text

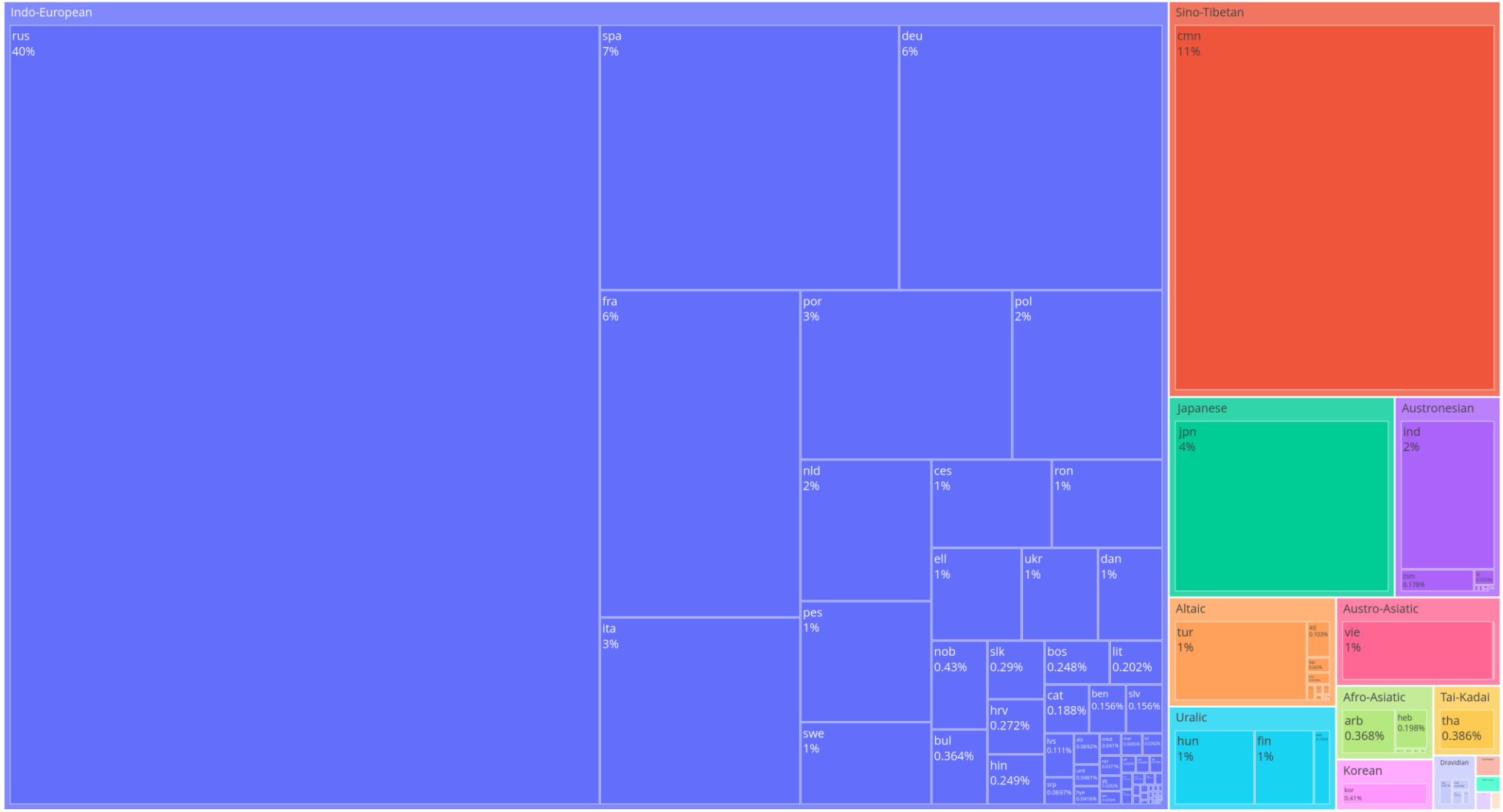
Pretraining data

- The most common approach for constructing publicly available pretraining datasets is to use Common Crawl dumps as a starting point
  - There as web crawls of publicly available websites
- Then various filtering and preprocessing steps are performed to extract text in the target langauge(s)
- The first pretraining datasets focussed mostly on English, but there has been an increase in efforts to build multilingual pretraining datasets
- Example datasets:
  - mC4, CC100
  - Glot500
  - CulturaX
  - Fineweb2
  - HPLT

Pretraining data distribution: The Pile



Multilingual pretraining data: HPLT 3.0: 13T tokens



Pretraining data preparation

- Remove boilerplate, quality filtering filter unsafe/adult content, deduplicate

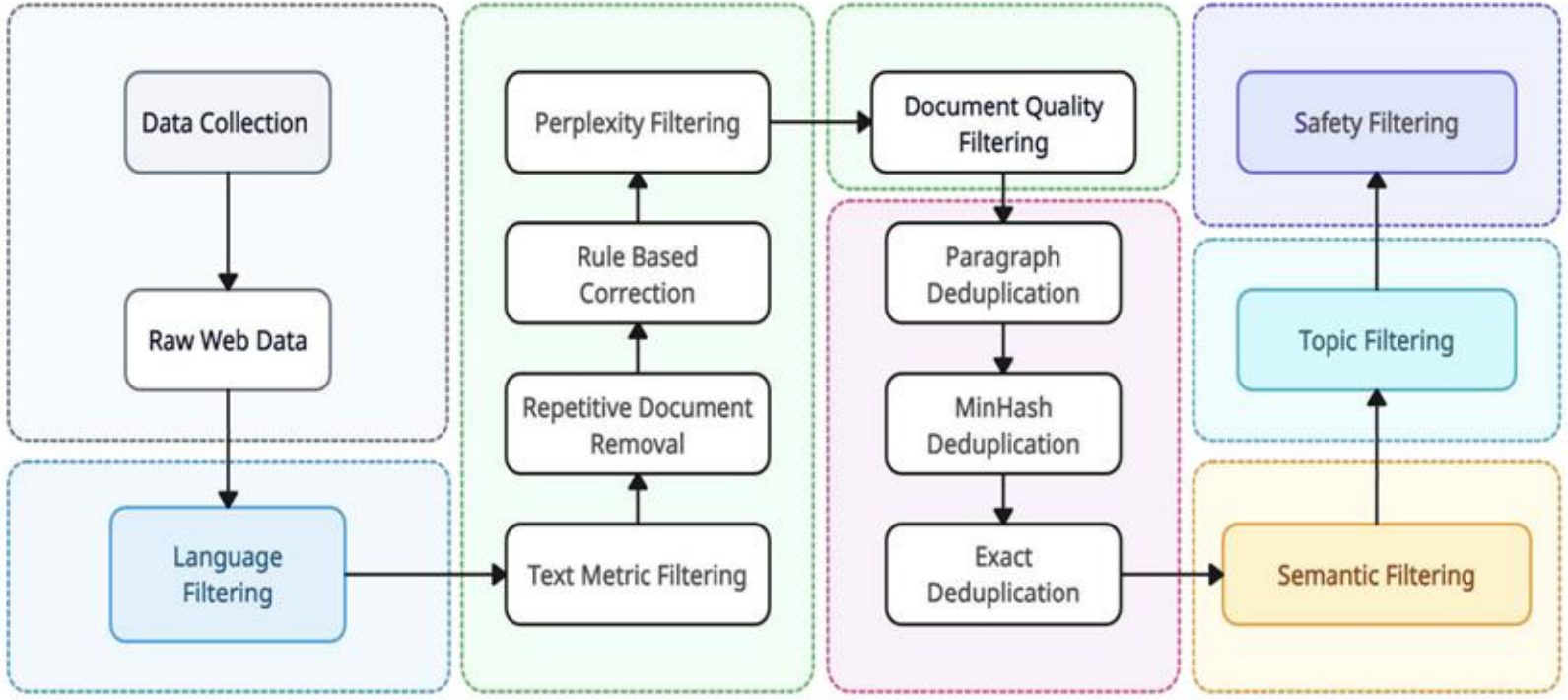
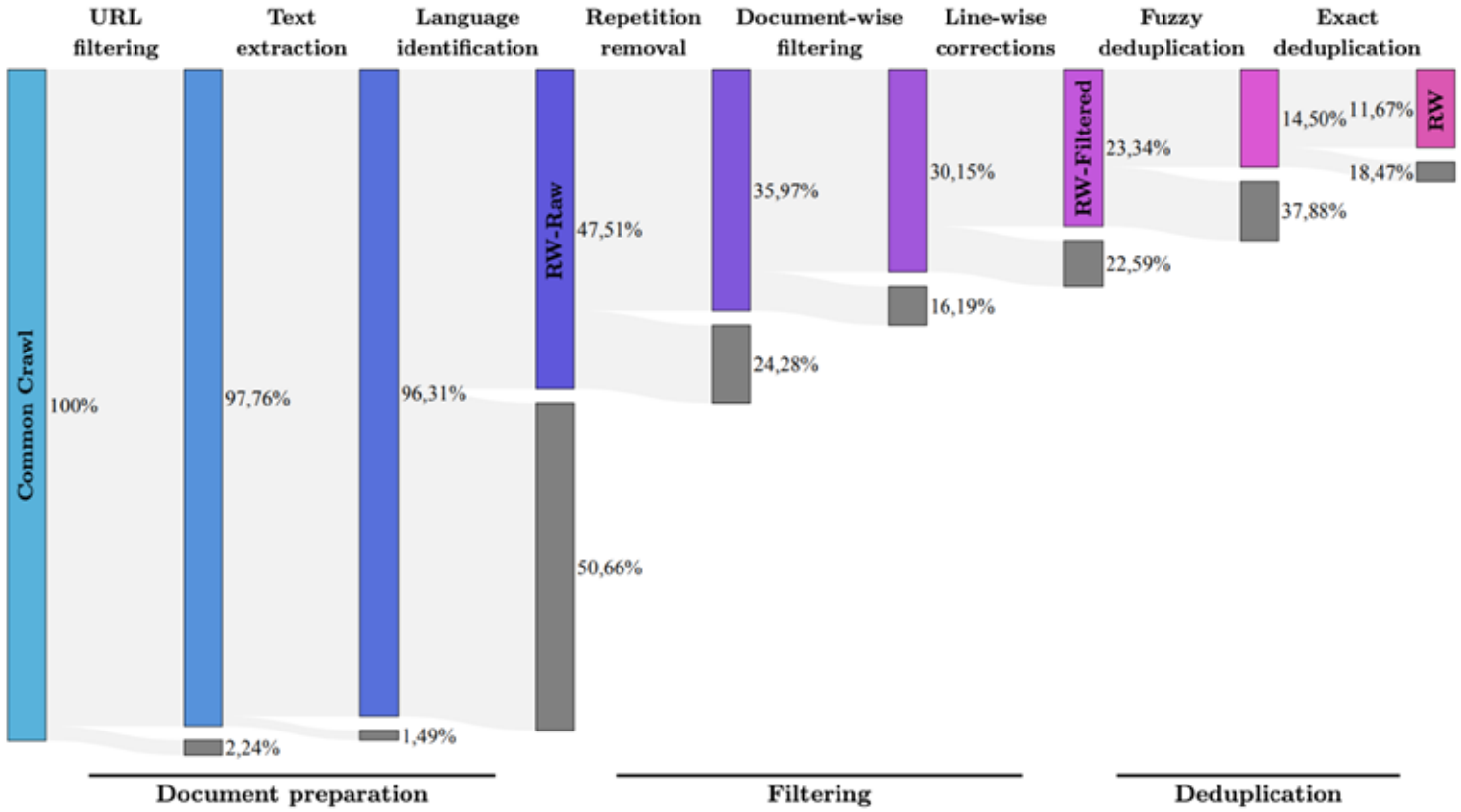


Figure 1: Yi's pretraining data cleaning pipeline.

Pretraining data preparation

- Data cleaning reduces the amount of data kept by 90%



Pretraining data: issues with web-scraped data

Copyright

- Much of the text in these datasets is copyrighted
- Open legal questions around legal usage of data

Data consent

- Website owners can indicate they don't want their site crawled

Privacy

- Websites can contain private IP addresses and phone numbers

Skew

- Training data is disproportionately generated by authors from the US which probably skews resulting topics and opinions

Pretraining data preparation

- Typical data-mix for open-source LLM pretraining: Olmo3

Source	Type	9T Pool		6T Mix		150B Mix	
		Tokens	Docs	Tokens	Docs	Tokens	Docs
Common Crawl	Web pages	8.14T	9.67B	4.51T (76.1%)	3.15B	121B (76.9%)	84.5M
olmOCR Science PDFs	Academic documents	972B	101M	805B (13.6%)	83.8M	19.9B (12.6%)	2.25M
StackEdu (Rebalanced)	GitHub code	137B	167M	409B (6.89%)	526M	11.1B (7.06%)	14.3M
arXiv	Papers with LaTeX	21.4B	3.95M	50.8B (0.86%)	9.10M	1.29B (0.82%)	247K
FineMath 3+	Math web pages	34.1B	21.4M	152B (2.56%)	95.5M	4.10B (2.60%)	2.57M
Wikipedia & Wikibooks	Encyclopedic	3.69B	6.67M	2.51B (0.04%)	4.24M	64.6M (0.04%)	119K
Total		**9.31T**	**9.97B**	**5.93T (100%)**	**3.87B**	**157B (100%)**	**104M**

Language model evaluation

Two approaches to evaluating language models:

- **Extrinsic evaluation** to compare models A and B:
  - Fine-tune each model for downstream task(s)
  - Run the task, get an accuracy for A and for B
  - Compare accuracy for A and B
- Extrinsic evaluation is expensive and time-consuming, so to develop language models we first do **intrinsic evaluation** using *perplexity*

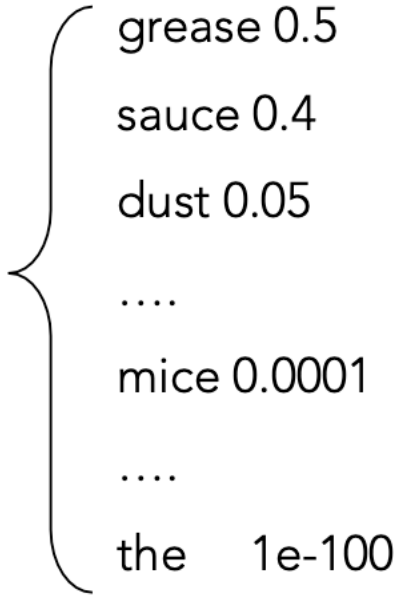
Evaluation

- How can we evaluate if a language model prefers good sentences over bad ones?
  - It should assign higher probability to "real" or "frequently observed" sentences than "ungrammatical" or "rarely observed" ones
- We train the model parameters on a training set
- We test the model's performance on a test set (ensure no overlap with training set)
- An evaluation metric tells us how well the model does on the test set

Evaluation

- The Shannon game: How well can we predict the next word?

When I eat pizza, I wipe off the ____
Many children are allergic to ____
I saw a ____



- A better language model assigns higher probability to the word that actually occurs

Evaluation

- The best language model is the one that assigns the highest probability to an unseen test set
- **Perplexity** is the inverse probability of the test set, normalized by the number of words

$$PPL(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

$$= \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \ldots w_{i-1})}}$$

Evaluation

An equivalent definition:

- Compute the per word log likelihood:

$$l = \frac{1}{M}\sum_{i=1}^{m} \log p(w_i | w_{1:i-1})$$

- Perplexity is

$$PPL(W) = 2^{-l}$$

- Minimizing perplexity is the same as maximizing probability

Evaluation

- Perplexity can be interpreted as a weighted branching factor

Examples:

- Recognizing digits '0,1,2,3,4,5,6,7,8,9'

$$\begin{aligned} \mathrm{PP}(W) &= P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}} \\ &= \left(\frac{1}{10}^{N}\right)^{-\frac{1}{N}} \\ &= \frac{1}{10}^{-1} \\ &= 10 \end{aligned}$$

- Recognizing (30,000) names
  - Perplexity = 30,000

Evaluation

Perplexity tells the story of language models

- Wall Street Journal corpus perplexity:

Uniform Distribution	10,000
Bigram Distribution	600
Trigram Distribution	200
5-gram + Tricks	140
Early Markov NNs	~140
Early non-Markov NNs	100

Slide: Sasha Rush

Evaluation

- Early results: perplexity is highly correlated with translation quality

perplexity

PPL (dev)	BLEU (dev)	params $\times 10^6$
4.92	25.8	65
5.29	24.9	
5.00	25.5	
4.91	25.8	
5.01	25.4	
5.16	25.1	58
5.01	25.4	60
6.11	23.7	36
5.19	25.3	50

Actual Translation Quality (higher is better)

Slide: Sasha Rush

Evaluation

- Later results: perplexity is highly correlated with everything

Hyperparams				Dev Set Accuracy		
#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2
3	768	12	5.84	77.9	79.8	88.4
6	768	3	5.24	80.6	82.2	90.7
6	768	12	4.68	81.9	84.8	91.3
12	768	12	3.99	84.4	86.7	92.9
12	1024	16	3.54	85.7	86.9	93.3
24	1024	16	3.23	86.6	87.8	93.7

perplexity

random NLP benchmarks

Slide: Sasha Rush

LLM Training

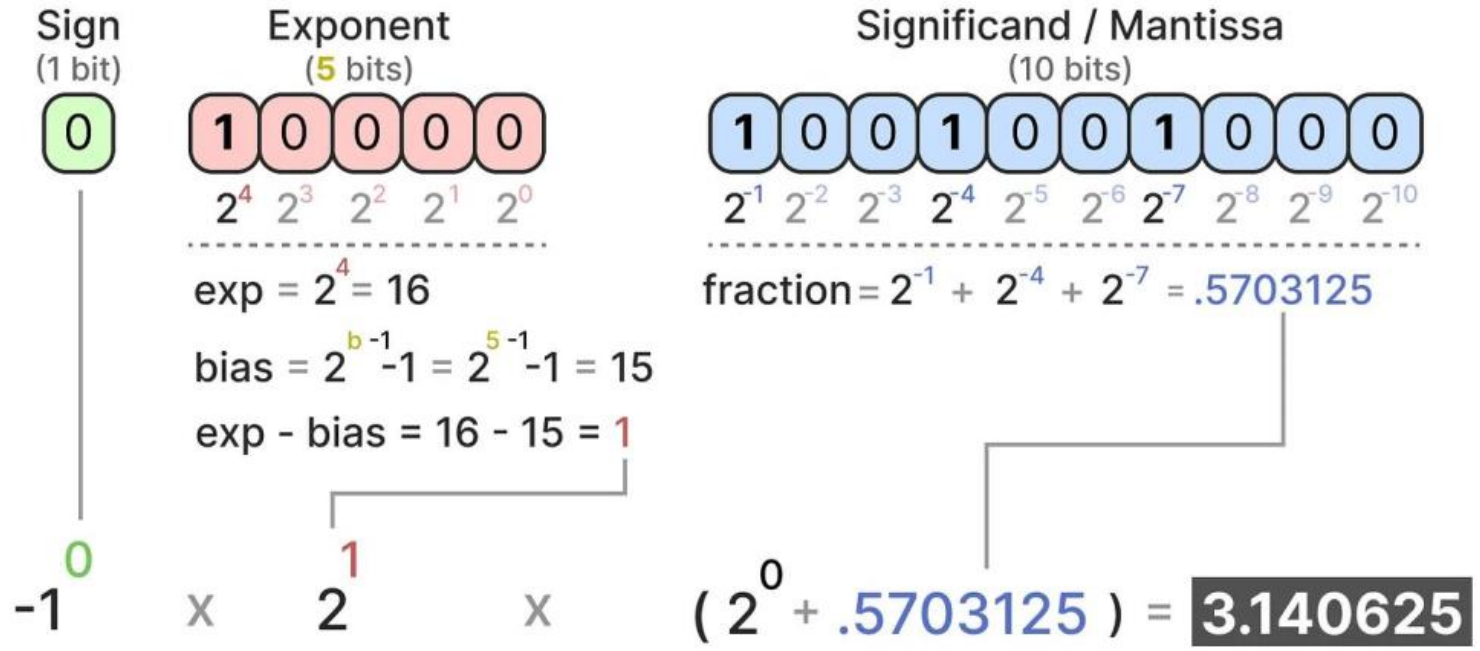
- Most of the large cost of developing LLMs goes into pretraining
- Performance needs to be optimized as much as possible and the available hardware used to its full capacity
- The first step is the optimize performance on a single GPU
- Large-scale training is performed on multiple GPUs using various forms of parallelism
- Most of these optimizations are outside the scope of this course, but we will mention a few relevant principles

Tensors

- Tensors are the basic building block for storing data, parameters, gradients, optimizer states, and activations
- Each tensor has a rank, which is its number of dimensions
- Elements of tensors are generally floating point numbers
- The choice of floating point representation influences both the speed and memory usage of a model

Floating point representations

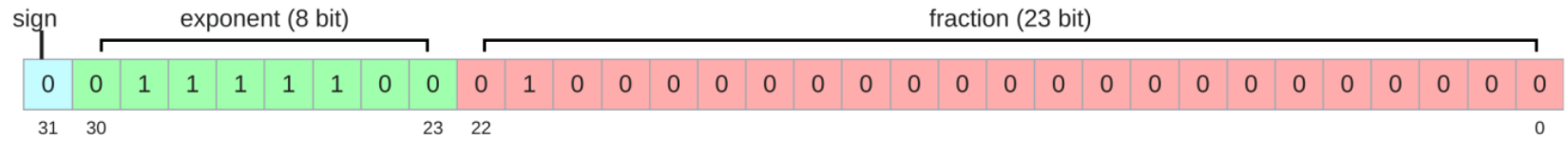
- Floating points consist of a sign bit, exponent bits (the range of numbers that can be represented), and the mantissa/significand (the precision – spacing between representable numbers)



Floating point representations

- fp32 (single precision) is the default, but we don't need the full precision and want to be as memory efficient as possible

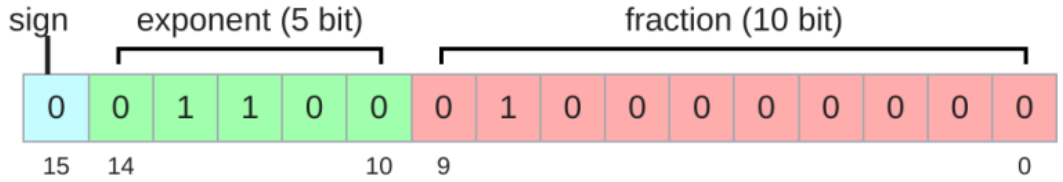
[Wikipedia]



- fp16 (half-precision) cuts down memory, but the dynamic range is limited

fp16

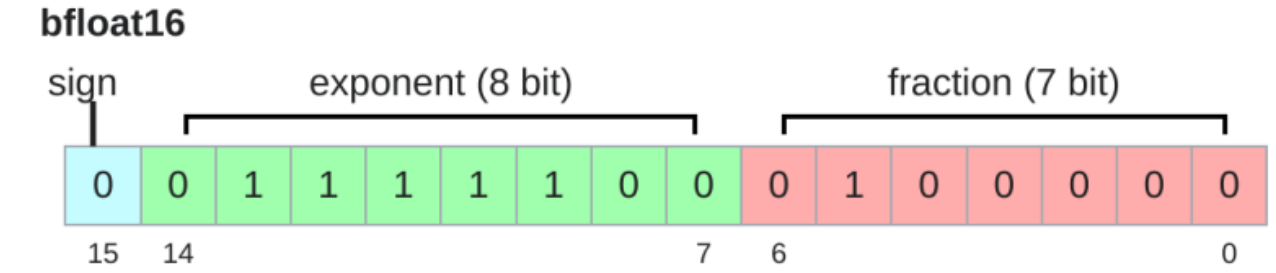
[Wikipedia]



Floating point representations

- bf16 (brain float) uses the same amount of memory as fp16 but maintains the full dynamic range of fp32, with worse precision

[Wikipedia]



- There is still some risk of floating point instability during training, but less than with fp16

Floating point representations

Practical solution is mixed precision:

- Use bf16 for parameters, activations, gradients
- Use fp32 for optimizer states, numerically sensitive operations – RMSNorm, softmax, loss computation, master copy of weights

- Even lower precision representations have been developed – 8 bit, 4 bit
- Lower precision increases inference efficiency, so trained models are often quantized to lower precision

Optimization

- As with most deep learning models, LLMs are training with some version of mini-batch Stochastic Gradient Descent (SGD)
- Various enhancements to vanilla SGD have been proposed, in particular to get more fine-grained control over the learning rate

Algorithm 7.2: Mini-batch stochastic gradient descent

Input: Training set of data points indexed by $n \in \{1, \ldots, N\}$
Batch size B
Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
Learning rate parameter η
Initial weight vector $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$
repeat
  $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{n:n+B-1}(\mathbf{w})$ `// weight vector update`
  $n \leftarrow n + B$
  if $n > N$ **then**
    shuffle data
    $n \leftarrow 1$
  end if
until convergence
return $\mathbf{w}$

Bishop and Bishop Deep learning

Optimization

- The **Adam** optimizer generally improves over standard SGD and is widely used in deep learning

Computes:

- *s*: exponential moving average of gradients (first moment)
- *r*: exponential moving average of squared gradients (second moment)
- Element-wide normalization of *s* by square root of *r* (adapts effective step size)

AdamW changes the weight decay behaviour of Adam

Algorithm 7.4: Adam optimization

Input: Training set of data points indexed by $n \in \{1, \ldots, N\}$
Batch size B
Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
Learning rate parameter η
Decay parameters β_1 and β_2
Stabilization parameter δ

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$
$\mathbf{s} \leftarrow \mathbf{0}$
$\mathbf{r} \leftarrow \mathbf{0}$
repeat
  Choose a mini-batch at random from $\mathcal{D}$
  $\mathbf{g} = -\nabla E_{n:n+B-1}(\mathbf{w})$ // evaluate gradient vector
  $\mathbf{s} \leftarrow \beta_1 \mathbf{s} + (1-\beta_1)\mathbf{g}$
  $\mathbf{r} \leftarrow \beta_2 \mathbf{r} + (1-\beta_2)\mathbf{g} \odot \mathbf{g}$ // element-wise multiply
  $\widehat{\mathbf{s}} \leftarrow \mathbf{s}/(1-\beta_1^\tau)$ // bias correction
  $\widehat{\mathbf{r}} \leftarrow \mathbf{r}/(1-\beta_2^\tau)$ // bias correction
  $\Delta\mathbf{w} \leftarrow -\eta \dfrac{\widehat{\mathbf{s}}}{\sqrt{\widehat{\mathbf{r}}} + \delta}$ // element-wise operations
  $\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w}$ // weight vector update
  $n \leftarrow n + B$
  if $n + B > N$ **then**
    shuffle data
    $n \leftarrow 1$
  end if
until convergence
return $\mathbf{w}$

Bishop and Bishop Deep learning

Batching

- The tokenized pretraining data is viewed as one long sequence with all documents concatenated, with an end-of-document token in-between.
- This sequence is split into subsequences of length *M*, which is the Transformer's context length.
  - The targets are the inputs shifted by one position
- Minibatches of size *B* are sampled from the data
- For single-GPU training, the size of the GPU memory (VRAM) limits sizes *M* and *B*

Batching

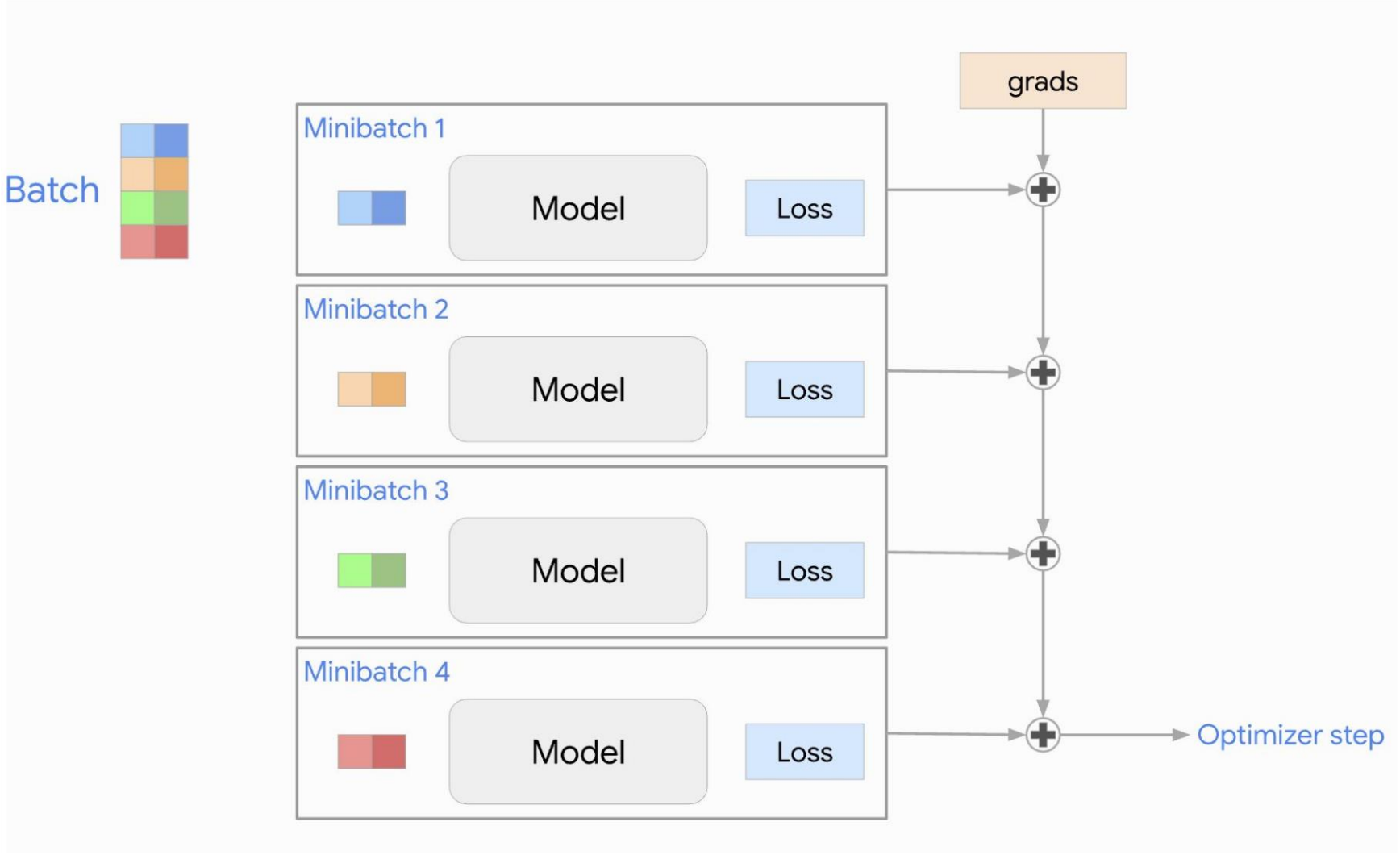
Estimated memory requirements for model training:

- Number of parameters $\approx 12\ L\ D^2$
- Parameter memory: 2 bytes / parameter (bf16)
- Gradient memory: 2 bytes / parameter (bf16)
- Optimizer state memory: 8 bytes / parameter (fp32 – first and second moments)
- Total: 12 bytes / parameter + activation memory
- Activation memory: $L\ S\ M\ B\ (34\ +\ 5)\frac{aM}{D}$
  - For *L* layers, model dimension *D*, *a* attention heads
  - Can be reduced through activation recomputation (rather than storing all forward activations in memory)

Gradient accumulation

- The default approach is to perform a weight update after computing the gradient for each mini-batch
- But large batch sizes often work better for LLM training
- Simulate the effect of a batch size larger than what fits in memory:
  - Calculate a running total of gradients for a predetermined number of accumulation steps *A*
  - Then perform a single optimizer step using the average of the accumulated gradients
  - Reset the gradient accumulator to zero to start next effective batch
  - This effective simulates an update based on *B***A* samples

Gradient accumulation



Multi-GPU training

Requires several parallelization approaches:

- Batch parallelism
- Data parallelism
- Fully sharded data parallelism
- Tensor parallelism
- Pipeline parallelism